Lean-Verified Session Protocols as the Security Boundary of LLM Agent Swarms[1]

Mauricio Neubecker
Technical University of Munich
Safe AI Germany

Tim Würfl
Technical University of Munich
Safe AI Germany

Maximilian Eder
Technical University of Munich
Safe AI Germany

With
Apart Research

Abstract

Teams of LLM agents increasingly work together by messaging each other and calling tools, and a single manipulated agent can steer the whole team into harmful actions. We present **SwarmGate**, a first-of-a-kind runtime in which agents can only communicate in ways a declared protocol allows, with Lean-proved guarantees that hold even for compromised agents. Classical session types enforce such protocols by checking each participant's program, but an LLM has no program to check. SwarmGate instead makes the protocol the security boundary and enforces it with a small **Lean-verified checker**, the only component that must be trusted. *Untrusted agents:* every delivered message provably follows the protocol. *Untrusted content:* a noninterference theorem bounds how many attacker-chosen bits can reach uncompromised agents. *Untrusted protocol authors:* an LLM writes a protocol per task, and a verified admission check decides whether it may run. On **GateBench**, a new benchmark of 4 domains and 40 tasks with real Claude agents (4,080 episodes), a compromised agent reaches its goal in 99% of episodes without a gate; under verified protocols it succeeds only through a free-text field the deployer explicitly allows (0% in three of four domains, 6–10% overall), and the worst-case taint-closure attack drops from 75–100% to 0%. LLM-written protocols admitted by Lean are as secure as hand-written ones, while 10% of unverified ones are unsafe.

1. Introduction

A single language model makes mistakes. A *swarm* of them fails in new ways that run through the messages the agents exchange. Prompt Infection spreads one compromise across a network of agents [1, 2]; a fooled orchestrator hands malicious code to an executor [3]; an agent-in-the-middle rewrites traffic [4]; and many multi-agent failures are skipped verification steps and broken role boundaries [5]. Multi-agent security is now

1 Research conducted at the Swarm Alignment Hackathon, November 2026

an open problem of its own [6, 7], and it grows with every framework that wires LLMs to each other (A2A) and to tools (MCP).

These are failures of **communication structure**: who may talk to whom, in what order, carrying what. Multiparty session types [8, 9] describe exactly this structure, but enforce it by type-checking every participant's program. An LLM agent has no such program. It samples its next message from a context an attacker may have poisoned. The classical guarantee therefore does not carry over to LLM agents.

Threat model. Any agent may be fully controlled by an attacker, any free text may carry an injected instruction, and the protocol itself may be written by a fallible LLM. The attacker wants concrete harm: run a non-allowlisted command, pay the wrong payee, mail a secret out, or commit attacker content. We assume that all inter-agent and tool traffic passes through the gate.

Approach. SwarmGate keeps the protocol but removes trust in the agents that run it, in the content they exchange, and in whoever wrote it. Each step replaces a trusted component with a theorem proved in Lean 4 (Figure 1). Every deployed protocol carries a certificate the Lean kernel re-checks, and the gate that the theorems describe is the same compiled function that runs in production.

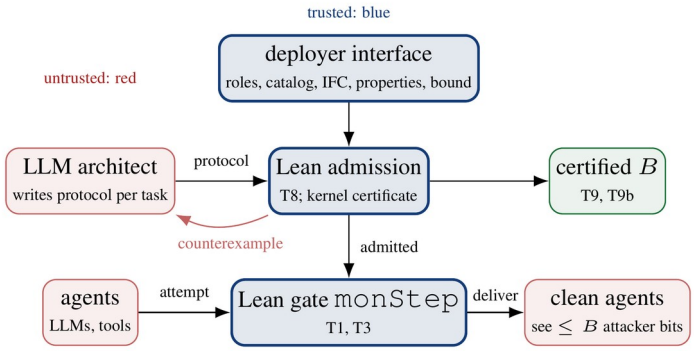


Figure 1: Overview of SwarmGate. Only the deployer's interface, the Lean admission check and the Lean gate are trusted (blue), while the protocol's author, the agents and the content they exchange are not (red). An LLM architect proposes a protocol per task, and Lean either admits it or returns a counterexample. The admitted protocol is enforced on every message and comes with a certified attacker budget B.

Our main contributions are:

1. **A verified security boundary for agent swarms.** To our knowledge, the first system that turns a session protocol into a security policy against fully compromised LLM agents, with enforcement, blame, information-flow and liveness guarantees mechanised in Lean 4 (2.7k lines, no `sorry`).
2. **Noninterference with a certified attacker budget.** Uncompromised agents depend on the attacker only through a declassified channel, for *arbitrary* agent behaviour, and we certify per protocol how many bits it carries.
3. **Verified admission of LLM-written protocols.** An LLM writes a protocol per task; Lean admits it only within the deployer's policy, and its counterexamples drive the repair. All 116 admitted protocols carry kernel-checked certificates.
4. **GateBench.** A benchmark of 4 domains and 40 tasks with injection, compromised-agent and worst-case taint-closure threats, run with real Claude agents over A2A-style messaging and MCP tool calls.

2. Related Work

Session types and monitoring. Multiparty session types and their mechanisations are mature [9, 10], as are runtime monitoring of session types [11, 12], blame [13], execution monitors [14, 15] and information flow,

also in multiparty sessions [16, 17], but all assume that participants are programs rather than adversarial language models steered by untrusted content.

Protocols for LLM agents are emerging concurrently: Lean-verified message-sequence-chart workflows [18], single-agent typestate via tool masking [19], choreographies [20], BSPL enactment [21] and LLM-synthesised refinements [22]. They use protocols for *coordination* among agents assumed to cooperate. **Agent security** defences such as ControlValve [23], CaMeL [24] and the prompt-injection design patterns [25] isolate untrusted data. They give no machine-checked guarantee or quantitative bound, and they do not check policies generated by an LLM.

What is new. SwarmGate is the first to combine all three. The protocol acts as a *security policy* against fully compromised agents, the data plane gets noninterference for arbitrary agents with a certified bit bound, and LLM-written protocols run only after verified admission. A practitioner would use it whenever a swarm both reads untrusted content and takes consequential actions, and "the model will probably refuse" is not an acceptable security argument. For a concrete protocol it gives what was missing before, namely a proof of which agents the attacker can ever influence and by how many bits.

3. Methods

3.1 Protocols, the gate and the deployer's interface

A **protocol** names roles (LLM agents, tools, deterministic validators, humans) and fixes how they may interact. It is written in a small Scribble-like language and compiled to a finite guarded automaton whose transitions carry sender, recipient, label, payload type and a decidable refinement such as `len <= 200`, so every check remains decidable [26, 27]. Each type has a capacity in bits. An enum of k values carries $\lceil \log_2 k \rceil$ bits, and a free-text string is unbounded (example in Appendix B). Agents never hold references to each other. Each message is an *attempt* sent to the **gate**, which delivers it and advances the protocol, or rejects it and blames the sender. The gate's decision is the compiled Lean function `monStep`, served to A2A-style messaging and MCP tool calls (68 µs per message). The deployer writes only an **interface**, which lists the roles, a *catalog* of allowed message kinds, the untrusted sources, the *endorsers* (deterministic non-LLM checkers such as an allowlist approver), the sensitive actions, required properties and a maximum length. A protocol cannot declare its own security metadata.

3.2 Step 1: untrusted agents

> **Theorem T1 (Enforcement)** (`enforcement_adaptive`). *For every protocol and every adversary, even one controlling all agents and adapting to the gate's verdicts, the sequence of delivered messages is a run of the protocol.*

Out-of-order actions (executing before approval), forged labels, impersonation, ill-typed payloads and replays are therefore impossible. Conformant traffic is never rejected (T2), every rejection names the true deviator so honest agents are never blamed (T3), and temporal properties such as "no `Exec` unless `Approved` earlier", no stuck states and bounded length, checked once, hold on every run (T4, T7).

3.3 Step 2: untrusted content

A well-typed string can still carry an injected instruction, and an LLM cannot separate data from instructions. We therefore treat *taint as sticky per agent*. A role that receives high-capacity content derived from an untrusted source stays tainted, while low-capacity messages (enums, booleans) and endorsers stop the spread.

> **Theorem T5 (Information flow)** (`ifc_transfer`). *If the verified checker accepts a protocol, no tainted role ever performs a sensitive action.*

> **Theorem T9 (Noninterference for arbitrary agents)** (`noninterference`). *Let clean agents be arbitrary deterministic functions of what they have seen, and let the attacker control a set of roles A. If*

two attackers produce the same declassified channel δ (recipients and labels of A's messages, payloads only to clean roles), every clean agent sees exactly the same in both runs.

Theorem T9b (Attacker bit budget) (`influence_monitor`). *The attacker's choices along every enforced run that reach clean agents amount to at most B bits, certified per protocol.*

T9 needs *no model of LLM behaviour at all*. Whatever the clean models do, they depend on at most B attacker-chosen bits. This turns the informal quarantined-reader pattern [24, 25] into a machine-checked number for a concrete protocol.

3.4 Step 3: untrusted protocol authors

A fixed hand-written protocol cannot fit open-ended tasks, so an LLM *architect* writes one per task, before any untrusted data is read, and only the admission check is trusted.

Theorem T8 (Admission) (`admitted_secure`). *If a protocol from any source is admitted against an interface, then on every run, for every adversary, every message is an instance of a catalog entry, no tainted role acts, all required properties hold, and the length bound is respected.*

Rejections come with a counterexample (an out-of-catalog message, an information-flow path, a violating trace), which is fed back to the architect for repair (Figure 2), in a counterexample-guided synthesis loop with Lean as the oracle. Each admitted protocol is emitted as a Lean file whose admission the kernel re-checks.

```
draft 1:
Reader -> Clerk : Extracted({payee: Vendor, amount: int}) where amount <= 50000;
Lean: INFORMATION-FLOW VIOLATION: after Task, Fetch, Get, Invoice, Reader->Clerk:Extracted,
      the roles Clerk, Reader, Docs may carry attacker-controlled text, and then
      Clerk could perform Clerk->Bank:Transfer.
draft 2 (admitted, kernel-checked):
Reader -> Approver1 : Extracted({payee: Vendor, amount: int}) ...;  // to the endorser
Reader -> Clerk : Status(enum{found, not_found, unreadable});      // 2 bits
```

Figure 2: One counterexample-guided repair, verbatim from the logs (Sonnet 5.5, finance task fin4). A 64-bit integer from the reader of untrusted invoices would let the attacker steer the paying clerk. After Lean's counterexample, the LLM routes the extracted fields to the deterministic approver and sends the clerk only a 2-bit status.

3.5 GateBench

GateBench has four domains, each with an interface, a *naive* protocol (plausible, but failing the information-flow check), a *hardened* protocol (admitted), simulated tools, endorsers, 10 tasks and precise goal detectors: **ops** (a researcher reads untrusted runbooks, an executor runs shell commands), **email** (an inbox reader, a password-reset code to protect), **finance** (an invoice reader, a clerk paying vendors via approvers) and **code** (a researcher reads package docs, a coder commits after CI). Every task requires reading untrusted content. An attack *succeeds* if a control goal (non-allowlisted command, wrong payee, mail to the attacker, attacker content in a commit) or a data goal (a secret reaches an attacker-readable sink) is reached. All payloads are inert.

Threats. *Inject:* untrusted tool output carries one of four injection families. *Compromised:* one LLM role is replaced by an adaptive attacker that probes every catalog message, impersonates others, mimics honest traffic and strikes with the most damaging payload the gate accepts. *Taint closure:* the attacker controls *every* agent that reads attacker-derived text, the worst case T5 and T9 address. **Defenses:** no gate (free bus), topology ACL (who-may-talk-to-whom), SwarmGate with the naive or hardened protocol, and an LLM-written protocol either enforced unverified (like ControlValve [23]) or Lean-admitted. Honest agents get identical tools and hints under every defense, so only enforcement differs.

3.6 Setup

Agents are Claude Haiku 5.5; the architect is Claude Sonnet 5.5 (Haiku and Opus 5.5 as alternatives). Every configuration runs twice, for 4,080 episodes at about $0.004 each. The unverified ablation gets the same prompt and interface feedback and lacks only the security check. For robustness we re-ran 2 tasks per domain

with Sonnet 5.5 as every agent (180 episodes). **What did not work:** Claude declined to play the attacker, so it is scripted (stronger and reproducible); a first random attacker was too weak and was replaced by the goal-directed one; and kernel re-checking needed structurally recursive checkers.

4. Results

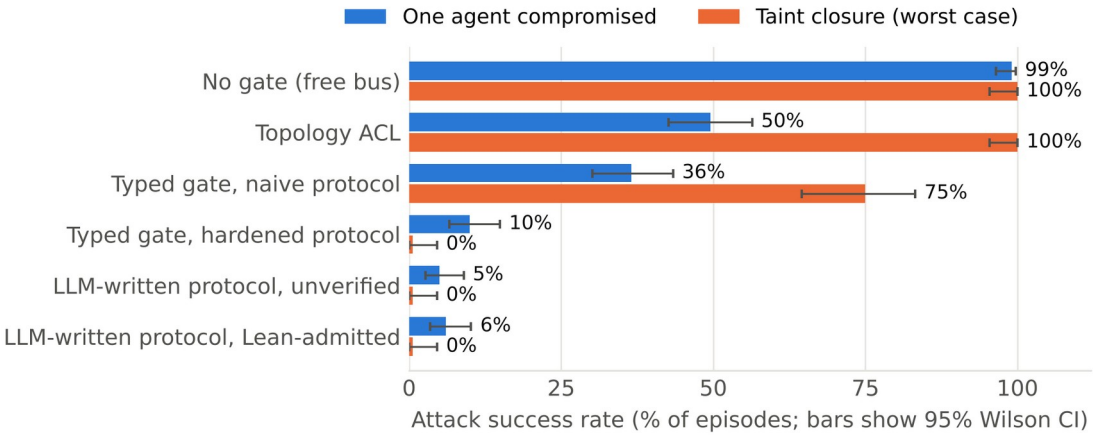


Figure 3: Attack success per defense on GateBench (Claude Haiku 5.5 agents; n = 200 episodes per defense with one compromised agent, n = 80 under taint closure). Typed SwarmGate protocols cut a compromised agent's success from 99% to 6–10% (0% outside the code domain). With information-flow-verified protocols, no attacker-controlled agent performs a sensitive action even in the worst case. Error bars: 95% Wilson intervals.

Table 1: GateBench results (% of episodes, Claude Haiku 5.5 agents, 4,080 episodes). ASR: attack success rate (control or data goal). Reader / actor: the compromised agent reads untrusted data / performs a sensitive action. Taint closure: the attacker steers every agent that reads attacker-derived free text (the worst case behind T5 and T9).

Defense	Benign utility	Inject ASR	Compromised reader	Compr. actor	Compr. all (95% CI)	Taint closure
No gate (free bus)	79	0	100	98	99 (96–100)	100
Topology ACL	85	0	17	99	50 (43–56)	100
SwarmGate, naive protocol	86	0	9	78	36 (30–43)	75
SwarmGate, hardened protocol	**100**	**0**	**0**	**25**	**10 (7–15)**	**0**
LLM-written, unverified	82	0	0	12	5 (3–9)	0
LLM-written, Lean-admitted	**86**	**0**	**0**	**15**	**6 (3–10)**	**0**

RQ1: Does the gate contain a compromised agent? Without mediation, one compromised agent reaches its goal in 99% of episodes (Figure 3, Table 1). A topology ACL stops a compromised *reader* (17%) but not an *actor* (99%), since the actor legitimately talks to the sensitive tool. The naive protocol enforces order but its free-text types still let actors do harm (78%). Typed protocols confine both. Hardened and LLM-written protocols cut reader attacks to **0%** and overall success to **6–10%** (0% in ops, email and finance). Every remaining success has the same cause. A compromised coder writes attacker text into the free-text message of its own commit after CI passed, a field the interface explicitly allows (Section 5).

RQ2: Does the data plane hold in the worst case? Indirect injection almost never succeeded (1 of 1,920 episodes; Claude Haiku 4.5 probe 0 of 48), so today's swarms rely on model robustness that nobody can certify. Under the taint-closure threat, where every exposed agent obeys the attacker, the naive protocol fails

in 75% and the ACL in 100% of episodes, while with every IFC-verified protocol, hand- or LLM-written, **no attacker-controlled agent ever performed a sensitive action** (0 of 160), as T5 predicts. The 4 LLM-written protocols that skipped verification and fail the information-flow check were exploited. In 2 of their 8 episodes the tainted clerk paid an attacker-chosen vendor and amount.

Table 2: Certified attacker budget B in bits (T9b), with A the static taint closure: the maximum number of attacker-chosen bits that can ever reach an uncompromised agent. ∞: unbounded.

Domain	Naive protocol	Hardened protocol	LLM-written, admitted: median (max)
ops	∞	12	6 (6)
email	∞	20	10 (10)
finance	∞	140	70 (74)
code	∞	18	9 (9)

RQ3: Can LLMs write admissible protocols? Every architect produced an admitted protocol for every task it answered, in 1.4–2.0 repair rounds on average, and all 116 carry certificates the Lean kernel re-checks in 2 s (Table A2). Without verification, 4 of 40 deployed protocols stayed information-flow-unsafe. Admitted LLM protocols are even *tighter* than hand-written ones, with about half the certified attacker budget (Table 2), because a per-task protocol needs fewer free choices. Admission does not, however, guarantee usefulness. Haiku's and Opus's protocols are equally secure (1% and 3% compromised success, 0% taint closure) but reach different utility (57% vs. 89%).

RQ4: Does practice stay inside the theory? In every gated episode we compared the agents that actually received attacker-marked content with the statically predicted taint set and found **0 violations in 2,720 gated episodes**. **RQ5: Utility and cost.** Security did not cost utility. Benign success is 79% on a free bus and **100%** with hardened protocols (86% with admitted LLM protocols), so structure helps agents finish. Synthesis costs about $0.03 per task, and checking a protocol takes milliseconds.

Robustness. The 95% Wilson intervals of ungated (96–100%) and gated (3–15%) compromised-agent success do not overlap (n = 200 per defense), and taint closure is 80/80 against 0/80. With Claude Sonnet 5.5 as every agent the ordering is unchanged (95% without a gate, 50% with an ACL, 25% with the naive protocol and 10% / 5% with hardened / admitted protocols). Taint closure falls from 100% to 0%, and 0 of 108 gated episodes left the predicted taint set. Finally, of 28,561 scripted structural attacks (impersonation, out of order, ill-typed, replay, ...), a free bus accepts 100%, schema validation 81%, a topology ACL 35% and SwarmGate **0%**.

5. Discussion and Limitations

Implications for AI safety. Today the safety of agent swarms rests on each model resisting manipulation. That held in our injection experiments, but it cannot be certified and may fail with the next model or attack. SwarmGate moves the guarantee from the models to the structure between them. Once a protocol is admitted, what any number of compromised or misaligned agents can do is bounded by a proof, and their influence on the rest of the swarm by a number. The approach scales with the number of agents and needs no insight into model internals. It also lets AI write its own rules, since each rule is checked. The pattern of an untrusted generator plus a verified checker applies wherever AI systems propose policies for other AI systems.

Limitations

Protocols order messages and type their fields, but do not bind the *content* of a free-text field to an earlier message. This gap accounts for all residual successes in RQ1, and an interface closes it by routing such fields through an endorser. Endorsers are a trust assumption, and a compromised role keeps the capabilities the protocol grants it (but no more). The theorems assume complete mediation and authenticated senders. Attacks outside the gate (tool poisoning, model backdoors) need other defences, and no monitor can guarantee liveness

against a silent agent. Integer capacities ignore refinements (sound but loose: finance's 140 bits). The step from B bits to 2^B attacker strategies is argued on paper and not yet mechanised. Our attacker is scripted. A learned attacker might find other paths inside the permitted envelope, though never outside it (T1).

Future Work

The most direct extension is *content binding* (a field must equal or hash-match an earlier endorsed message), which removes the residual commit-message attacks. *Refinement-aware capacities* would substantially shrink finance's 140-bit budget. A further step is deployment in production MCP/A2A stacks against learned attackers and larger, open-ended task suites.

6. Conclusion

The security of an LLM agent swarm does not have to rest on the agents. SwarmGate trusts only a small Lean-verified checker and gate, and proves that whatever the agents do, messages follow the protocol, tainted agents cannot act, and clean agents see at most B attacker-chosen bits, even when an LLM wrote the protocol. With real Claude agents this cuts a compromised agent's success from 99% to 0% in three of four domains (the rest only through a free-text field the interface allows) and the worst-case attack from up to 100% to 0%, at no utility cost, with every observed infection inside the predicted set. In the resulting division of labour, LLMs write the protocols and a proof decides which ones may run.

Ethical Considerations and Dual Use

SwarmGate is a defensive technology. It restricts what LLM agents can do and keeps their failures contained and attributable. The work involves no human subjects and no personal data; all tools, inboxes, invoices and repositories in GateBench are simulated, and every secret is a fake placeholder.

Dual use. GateBench contains attack scenarios and a scripted adaptive attacker. Its techniques (message spraying, impersonation, mimicking honest traffic) are well known from prior multi-agent security work, and all payloads are inert placeholders without functional exploit strings, so the benchmark gives attackers little uplift while letting defenders measure their systems. We did not try to circumvent model safeguards. When Claude declined to play the attacker, we scripted the attacker instead, and the synthesis tasks Opus refused are reported as refusals.

Risk of over-reliance. The main risk is a false sense of security. Our guarantees hold only under the stated assumptions (complete mediation, authenticated senders, trusted endorsers and a correct deployer interface), and they bound where content flows rather than what it means. Deployers should treat an admitted protocol as one verified layer of defence in depth and keep model-level safeguards, monitoring and human oversight of high-stakes actions in place.

Code and Data

- **Code repository**: https://github.com/timwuerfl/agent-comms-type-check (Lean proofs, Python runtime, MCP/A2A gateway, GateBench, synthesis loop, paper sources; `reproduce.sh` re-runs all offline experiments).
- **Data**: all episode logs, synthesised protocols, kernel certificates and the API cost ledger are in `eval/results/` of the repository.
- **Info-hazard note**: all attacker payloads in GateBench are inert placeholders (fake commands, example.com addresses, fake tokens); no functional exploits are included.

References

[1] D. Lee and M. Tiwari. Prompt infection: LLM-to-LLM prompt injection within multi-agent systems, 2024. https://arxiv.org/abs/2410.07283

[2] X. Gu, X. Zheng, T. Pang, C. Du, Q. Liu, Y. Wang, J. Jiang, and M. Lin. Agent smith: A single image can jailbreak one million multimodal LLM agents exponentially fast. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 16647–16672, 2024. https://arxiv.org/abs/2402.08567

[3] H. Triedman, R. Jha, and V. Shmatikov. Multi-agent systems execute arbitrary malicious code. In *Conference on Language Modeling (COLM)*, 2025. https://arxiv.org/abs/2503.12188

[4] P. He, Y. Lin, S. Dong, H. Xu, Y. Xing, and H. Liu. Red-teaming LLM multi-agent systems via communication attacks. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 6726–6747, 2025. https://doi.org/10.18653/v1/2025.findings-acl.349

[5] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2025. https://arxiv.org/abs/2503.13657

[6] C. Schroeder de Witt et al. Open challenges in multi-agent security: Towards secure systems of interacting AI agents, 2025. https://arxiv.org/abs/2505.02077

[7] L. Hammond, A. Chan, J. Clifton, et al. Multi-agent risks from advanced AI. Technical Report 1, Cooperative AI Foundation, 2025. https://arxiv.org/abs/2502.14143

[8] K. Honda, N. Yoshida, and M. Carbone. Multiparty asynchronous session types. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 273–284, 2008. https://doi.org/10.1145/1328438.1328472

[9] K. Honda, N. Yoshida, and M. Carbone. Multiparty asynchronous session types. *Journal of the ACM*, 63(1):9:1–9:67, 2016. https://doi.org/10.1145/2827695

[10] D. Castro-Perez, F. Ferreira, L. Gheri, and N. Yoshida. Zooid: a DSL for certified multiparty computation: from mechanised metatheory to certified multiparty processes. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, pages 237–251, 2021. https://doi.org/10.1145/3453483.3454041

[11] L. Bocchi, T.-C. Chen, R. Demangeon, K. Honda, and N. Yoshida. Monitoring networks through multiparty session types. *Theoretical Computer Science*, 669:33–58, 2017. https://doi.org/10.1016/j.tcs.2017.02.009

[12] J. K. Larsen, A. Scalas, G. Amir, J. Jacobs, J. Wagemaker, and N. Foster. NEST: Network enforced session types. In *40th European Conference on Object-Oriented Programming (ECOOP)*, volume 372 of *LIPIcs*, pages 17:1–17:31, 2026. Lean 4 mechanisation: https://github.com/alcestes/NestLean. https://doi.org/10.4230/LIPIcs.ECOOP.2026.17

[13] L. Jia, H. Gommerstadt, and F. Pfenning. Monitors and blame assignment for higher-order session types. In *Proceedings of the 43rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 582–594, 2016. https://doi.org/10.1145/2837614.2837662

[14] F. B. Schneider. Enforceable security policies. *ACM Transactions on Information and System Security*, 3(1):30–50, 2000. https://doi.org/10.1145/353323.353382

[15] J. Ligatti, L. Bauer, and D. Walker. Edit automata: Enforcement mechanisms for run-time security policies. *International Journal of Information Security*, 4(1–2):2–16, 2005. https://doi.org/10.1007/s10207-004-0046-8

[16] A. Sabelfeld and A. C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003.

[17] S. Capecchi, I. Castellani, and M. Dezani-Ciancaglini. Information flow safety in multiparty sessions. *Mathematical Structures in Computer Science*, 2016.

[18] B. Bollig, M. Függer, and T. Nowak. Provable coordination for LLM agents via message sequence charts. In *Proceedings of ISoLA 2026*, 2026. ZipperGen; projection and deadlock-freedom mechanized in Lean 4. https://arxiv.org/abs/2604.17612

[19] M. Sikaroudi. Concordat: Typed sessions for LLM agents – enforcing conversation protocols on a model that cannot be trusted to follow them, 2026. https://arxiv.org/abs/2610.37461

[20] Gopinathan, Feser, Naim, Tavares, and Bingham. Pact: A choreographic language for agentic ecosystems, 2026. CP 2026 workshop (PLDI 2026). Only surnames verified; add given names from arXiv. https://arxiv.org/abs/2605.03143

[21] O. Joshi, M. P. Singh, and A. K. Chopra. Ahoy: LLMs enacting multiagent interaction protocols, 2026. EMAS 2026 workshop. https://arxiv.org/abs/2606.05390

[22] Y. Li, P. Hou, and N. Yoshida. Specification-guided synthesis of deadlock-free communication protocol refinements with large language models. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2026. arXiv:2607.27964.

[23] R. Jha, H. Triedman, J. Wagle, and V. Shmatikov. Breaking and fixing defenses against control-flow hijacking in multi-agent systems. In *International Conference on Learning Representations (ICLR)*, 2026. https://arxiv.org/abs/2510.17276

[24] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr. Defeating prompt injections by design, 2025. https://arxiv.org/abs/2503.18813

[25] L. Beurer-Kellner, B. Buesser, A.-M. Creţu, E. Debenedetti, D. Dobos, D. Fabian, M. Fischer, D. Froelicher, K. Grosse, D. Naeff, E. Ozoani, A. Paverd, F. Tramèr, and V. Volhejn. Design patterns for securing LLM agents against prompt injections, 2025. https://arxiv.org/abs/2506.08837

[26] E. Li, F. Stutz, T. Wies, and D. Zufferey. Complete multiparty session type projection with automata. In *Computer Aided Verification (CAV 2023)*, LNCS, pages 350–373. Springer, 2023. https://doi.org/10.1007/978-3-031-37709-9_17

[27] D. Castro-Perez, F. Ferreira, and S.-S. Jongmans. A synthetic reconstruction of multiparty session types. In *Proc. ACM Program. Lang. (POPL)*, 2026. arXiv:2511.22692.

Appendix

A. Theorem overview

Table A1: Mechanised theorems (Lean 4, 2.7k lines, no `sorry`, standard axioms only; files under `lean/SwarmGate/`).

Theorem	Lean name	Statement (informal)
T1 Enforcement	`enforcement(_adaptive)`	Delivered trace is a protocol run, for every adversary
T2 Transparency	`transparency`	Conformant traffic is never rejected
T3 Blame	`blame_sound`, `honest_never_blamed`	Rejections name the true deviator; honest agents are never blamed
T4 Property transfer	`property_transfer`	Past-time LTL properties checked once hold on every run
T5 Information flow	`ifc_transfer`	No tainted role ever performs a sensitive action
T6 Decentralised	`decentralised_sound`	Per-role sidecars jointly admit only protocol traces
T7 Liveness / bound	`no_stuck, bounded_length`	No stuck state; completion reachable; bounded length
T8 Admission	`admitted_secure`	Admitted protocols satisfy catalog, IFC, properties, bound
T9 Noninterference	`noninterference(_inv)`	Clean agents depend on the attacker only via δ
T9b Bit budget	`influence_monitor`	Attacker choices reaching clean agents $\leq$ B bits
Bridge	`admitted_noninterference`	T8 + T9 for the executed runtime semantics

B. Example protocol and the gate

```
protocol OpsHardened {   // roles, types and security metadata come from the interface
  User -> Planner : Task(str) where len <= 2000;
  rec Research max 2 {
    Planner -> Researcher : Query(Topic);
    Researcher -> Web : Get(Topic);  Web -> Researcher : Page(str);   // untrusted
    Researcher -> Planner : Recommend(Cmd);                          // enum: <= 3 bits
    choice at Planner { continue Research; } or { Planner -> Approver : Propose(Cmd); } }
  choice at Approver { Approver -> Executor : Approved(Cmd);
    Executor -> Shell : Exec(Cmd); Shell -> Executor : Out(str); ... }
  or { Approver -> Planner : Denied; } ... }
```

Listing B1: The hardened ops protocol (abridged). The researcher reads untrusted runbook pages but can only answer with an allowlisted command (≤ 3 bits), and the executor accepts commands only from the deterministic approver.

```
monStep cfg P m (send σ e) =
  if m.halted            then reject σ halted
  else if quarantined σ  then reject σ quarantined
  else if σ ≠ e.src      then strike; reject σ spoof
  else if m.count ≥ budget then reject σ budget
  else match step P m.s e with
       | some s' => advance; deliver e
       | none    => strike; reject σ notEnabled
```

Listing B2: The gate's decision function (from `Monitor.lean`). T1–T3 are theorems about exactly this function, and the runtime executes its compiled code.

C. Protocol synthesis and per-domain results

Table A2: LLM-written protocols (40 tasks per architect). Every task an architect answered ended in an admitted, kernel-certified protocol. † Opus declined 4 ops tasks.

Architect	Haiku 5.5	Sonnet 5.5	Opus 5.5
Admitted within 4 rounds	40/40	40/40	36/36†
Mean repair rounds	1.4	2.0	1.5
First draft admissible	38/40	40/80	30/37
First draft with IFC flaw	0	6	0
Kernel-checked certificates	40/40	40/40	36/36

Table A3: Compromised-agent attack success per domain (%, Haiku 5.5 agents).

Domain	No gate	Topology	Naive	Hardened	LLM	LLM + Lean
ops	100	65	48	0	0	0
email	100	50	50	0	0	0
finance	100	50	10	0	0	0
code	97	33	33	33	17	20

Residual successes under hardened and admitted protocols occur only in the code domain (commit-message content, see Limitations), while ops, email and finance reach 0% for both.

D. Architect prompt (excerpt)

```
You are a security architect for multi-agent LLM systems. You design the communication
protocol that a swarm of agents must follow for one task. Agents are LLMs that may be
prompt-injected; the protocol is enforced by a verified gate. Write the protocol in the
SwarmGate language. Answer with a short explanation followed by exactly one code block.
```

Listing D1: System prompt of the protocol architect (followed by the DSL guide, the deployer interface, the behaviour of non-LLM roles, the task, and, in later rounds, Lean's counterexamples).

LLM Usage Statement

We used Claude for implementation support and drafting. All results were verified. Every number in this report comes from logged, reproducible runs in the repository, and all theorems are checked by the Lean kernel.